

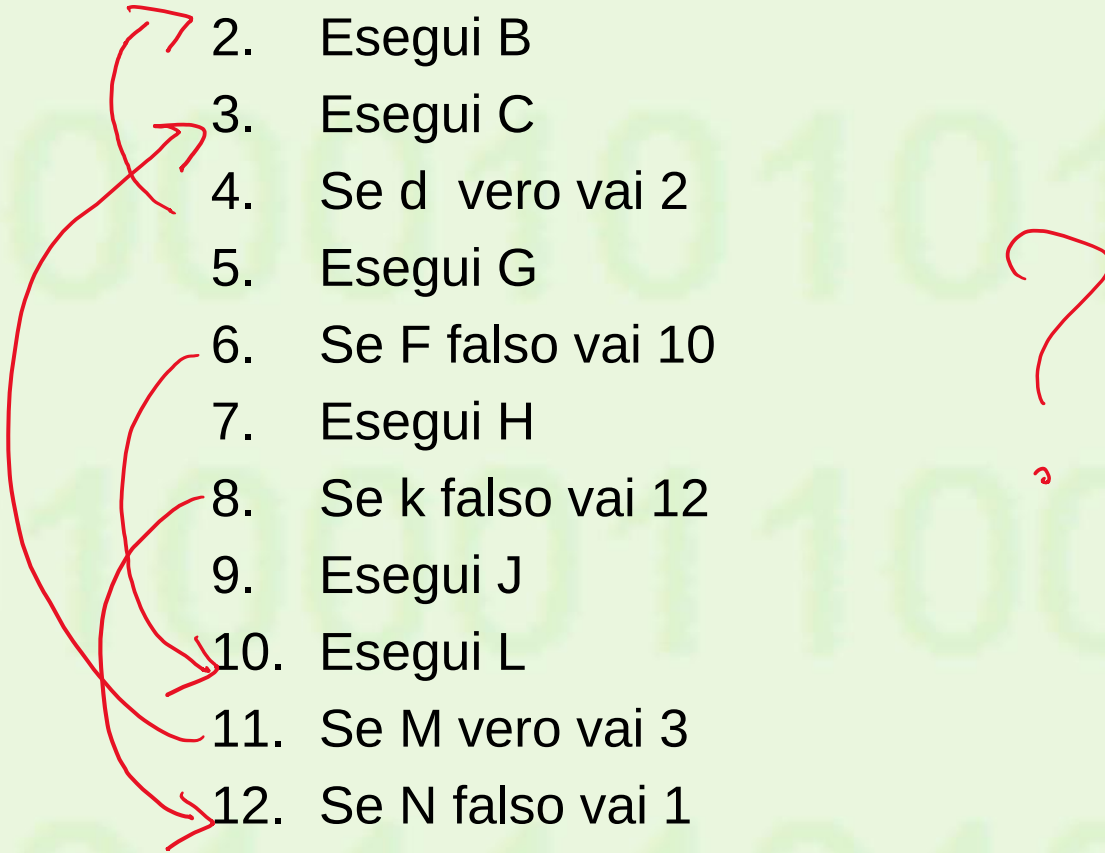
Laboratorio di Programmazione

(Laurea triennale in matematica)

Lezione 5

Esempio di programma con goto

1. Esegui A
2. Esegui B
3. Esegui C
4. Se d vero vai 2
5. Esegui G
6. Se F falso vai 10
7. Esegui H
8. Se k falso vai 12
9. Esegui J
10. Esegui L
11. Se M vero vai 3
12. Se N falso vai 1
13.
14.



Programmazione strutturata

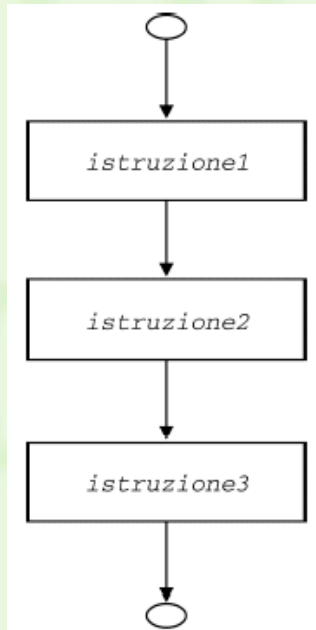
Paradigma di programmazione che fa uso di:

- **Strutture di controllo:**
 - Sequenza (succesione finita di istruzioni);
 - Selezione;
 - Iterazione/ricorsione
- **Componibilità delle strutture di controllo**

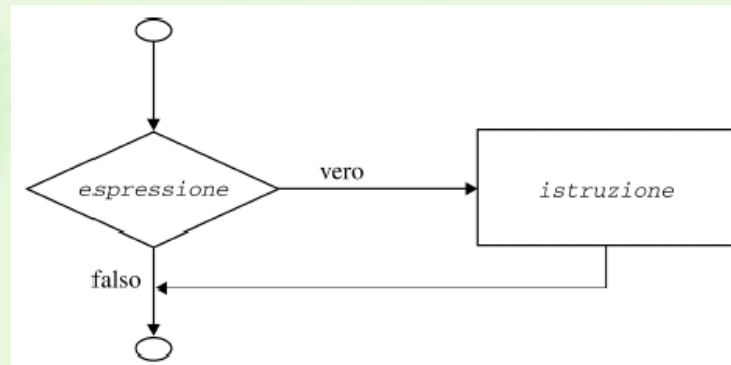
L'obiettivo della *programmazione strutturata* è di rendere un flusso ordinato il passaggio tra le istruzioni dall'inizio alla fine dei programmi.

Teorema di Böhm- Jacopini

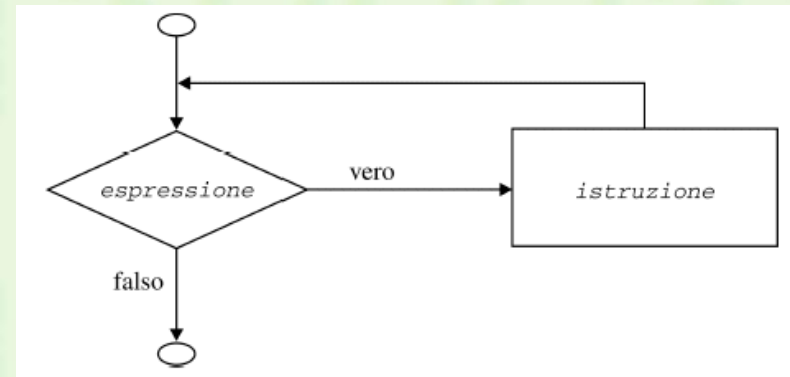
Corrado Böhm e Giuseppe Jacopini hanno dimostrato che tutti i programmi possano essere scritti con l'utilizzo di: **sequenza**, **selezione** e **iterazione** (senza l'uso del salto *goto*).



Sequenza



Selezione



Iterazione

Comandi condizionali o di selezione

- Esprimono **un'alternativa** fra due o più possibili prosecuzioni della computazione sulla base della *valutazione di una espressione logica*

Comandi iterativi

- Permettono di esprimere computazioni che si *ripetono* un numero di volte che è variabile.
- **Iterazione determinata**: con costrutti che permettono di iterare un *numero fissato di volte*;
- **Iterazione indeterminata**: con costrutti che permettono di iterare fino al *verificarsi di una determinata condizione*;

(La ricorsione)

- Metodo alternativo all'iterazione in cui un processo di calcolo richiama se stesso (la vedremo tra un po' di lezioni...)

L'istruzione **if**

- Sintassi

```
if ( <espressione booleana> ){  
    <istruzione_1>;  
    .....  
    <istruzione_r>;  
}  
  
<istruzione_x>;  
<istruzione_y>;  
.....
```

- Significato

Il blocco <istruzione_1>..*<istruzione_r>* viene eseguito **soltanto** se <espressione booleana> è *true*

Istruzione **while**

- Sintassi

```
while (<espressione booleana>){  
    <istruzione_1>  
    ...  
    <istruzione_r>  
}  
  
<istruzione_x>;  
<istruzione_y>;  
.....
```

- Significato

Il blocco <istruzione_1>..**<istruzione_r>** viene eseguito **fintanto che** <espressione booleana> è **true**

Iterazione definita : ciclo con "contatore"

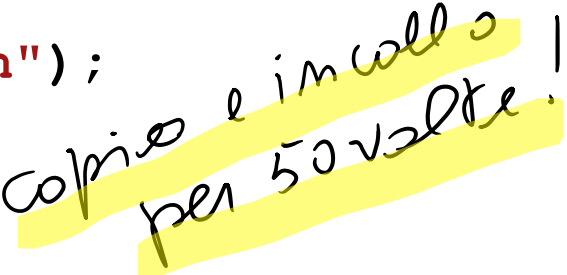
numero di iterazioni conosciuto a-priori

- Utilizzo una variabile per contare:
 - La inizializzo a 0 prima del ciclo
 - All'interno del ciclo la incremento
 - La condizione del while controlla quando arrivo al valore scelto

```
cont =0;
while (cont < 3)
{
    printf("Tutto sbagliato!");
    cont= cont+1;
}
prezzo = 50;
.....
```

Esercizio: Calcolare la somma di 50 numeri immessi dall'utente.

```
int a, s;  
s=0;  
printf("scrivi il numero\n");  
scanf("%d", &a);  
s=s+a;  
  
printf("scrivi il numero\n");  
scanf("%d", &a);  
s=s+a;  
  
printf("scrivi il numero\n");  
scanf("%d", &a);  
s=s+a;  
.....  
.....  
printf("la somma dei numeri è %d", s);
```



Esercizio : Calcolare la somma di 50 numeri immessi dall'utente.

muovo!

```
int a, s, c;
```

```
s=0;
```

```
c=0;
```

```
while( c < 50 ){
```

```
    printf("scrivi il numero\n");
```

```
    scanf("%d", &a);
```

```
    s=s+a;
```

```
    c=c+1;
```

```
}
```

```
printf("la somma dei numeri è %d", s);
```

c è il mio contatore

Un ciclo a iterazione "indefinita"

numero di iterazioni NON conosciuto a-priori

- La condizione del while determina il numero di volte che il ciclo viene eseguito .. (può dipendere dalla particolare esecuzione)

```
c=1;
scanf ("%d", &num) ;
while (num !=33)
{
    printf(" \n No, non hai indovinato ...);
    c=c+1;
    printf(" \n Inserisci un altro numero :");
    scanf ("%d", &num) ;
}
printf("hai indovinato il mio numero in %d
tentativi", c) ;
```

Esercizio: Calcolare la somma di vari numeri immessi dall'utente; l'immissione termina quando l'utente inserisce 0

```
int a, s;

printf("scrivi il numero\n");
scanf("%d", &a);
s=0;

while( a != 0 ){
    s=s+a;
    printf("scrivi il numero\n");
    scanf("%d", &a);
}

printf("la somma dei numeri è %d", s);
```

Attenzione.... Errori in vista!

```
int num = 5;
while( num > 3 ){
    printf("hello world");
}
```

NO

```
int num = 5;
while( num > 3 ){
    printf("hello world");
    num=num+1;
}
```

NO

```
int num = 5;
while( num < 7 ){
    printf("hello world");
    num=num+1;
}
```

OK

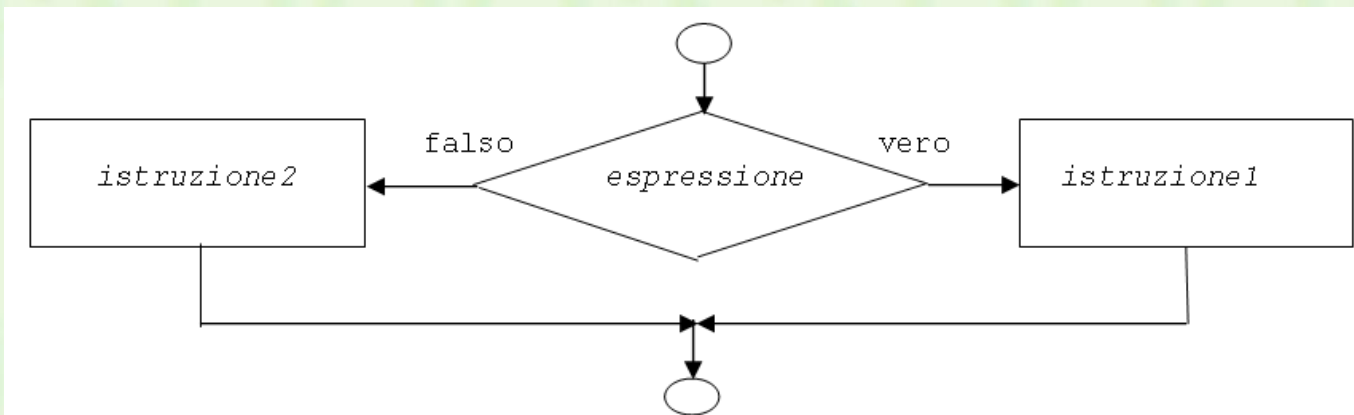
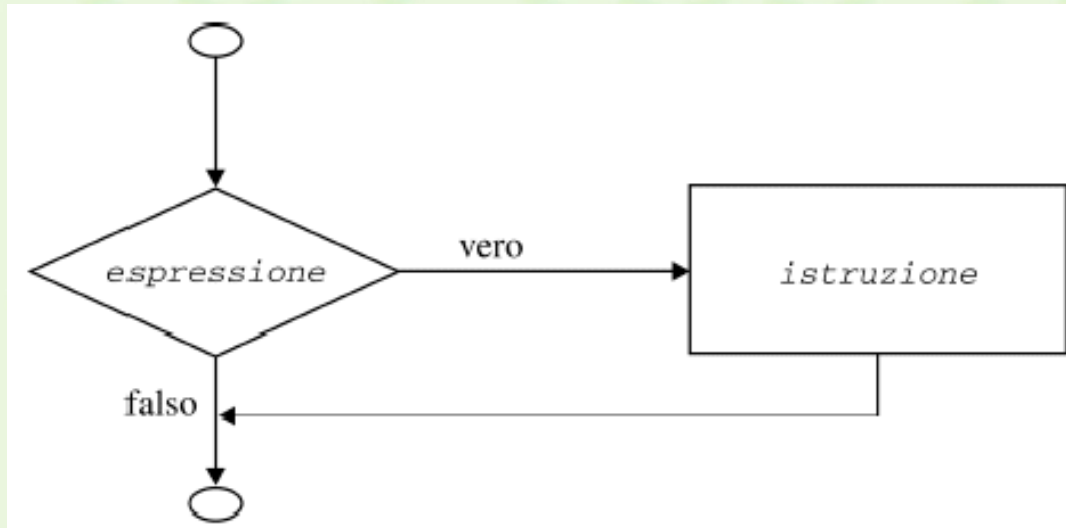
Istruzione **while**

ATTENZIONE!!!

Tra le istruzioni del corpo del **while** ci deve essere qualcosa che coinvolge e modifica le variabili della *<espressione booleana>* e la rende false

Altrimenti... LOOP INFINITO

Due possibili schemi per la selezione:



L'istruzione **if**

- Sintassi

```
if ( <espressione booleana> ){  
    <istruzione_1>;  
    .....  
    <istruzione_r>;  
}  
<istruzione_x>;  
<istruzione_y>;  
.....
```

- Significato

Il blocco <istruzione_1>..*<istruzione_r>* viene eseguito **soltanto** se <espressione booleana> è *true*

L'istruzione **if...else**

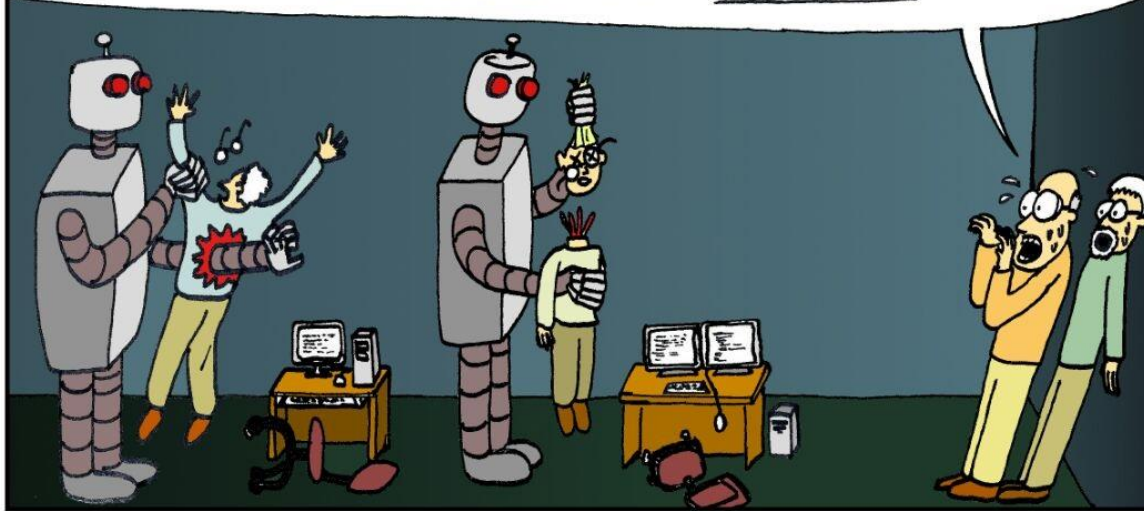
- Sintassi

```
if ( <espressione booleana> )  
    { <istruzione_1>;  
      .....  
      <istruzione_r>; }  
else  
    { <istruzione_r+1>;  
      .....  
      <istruzione_s>; }  
<istruzione_x>;  
<istruzione_y>;  
.....
```

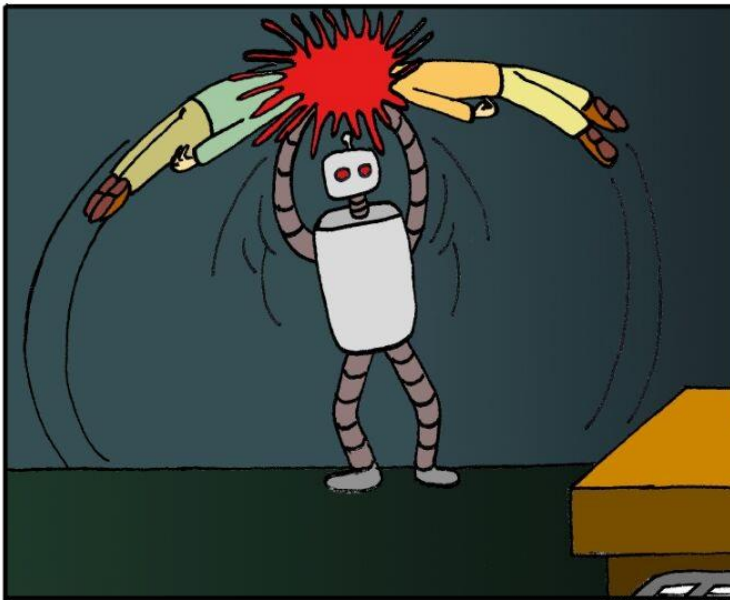
- Significato

SE *<espressione booleana>* è **true** viene eseguito Il blocco
<istruzione_1>...<istruzione_r> **ALTRIMENTI** viene eseguito Il blocco
<istruzione_r+1>...<istruzione_s>;

OH NO! THE ROBOTS ARE KILLING US!!!



BUT WHY?!? WE NEVER PROGRAMMED THEM TO DO THIS!!!



```
main() {  
    int isCrazyMurderingRobot=0;  
  
    if (isCrazyMurderingRobot=1) {  
        kill(humans);  
    }  
    else  
        be_nice_to(humans);  
}
```

Esercizio

```
if (a%3==0 || a%2==0)
    printf("%d", 2*a);
else
    printf("%d", 5*a);
```

- Se a=10 il programma stampa ?
- Se a=42 il programma stampa ?
- Se a=18 il programma stampa ?
- Se a=25 il programma stampa ?

Esercizio

```
int m, a, ms, as;
if (m==12) {
    ms = 1;
    as = a + 1;
    printf("%d,%d", ms,as);
}
else {
    if (m<12) {
        ms = m + 1;
        as = a;
        printf("%d,%d", ms,as);
    }
    else
        printf("%d,%d", m,a);
}
```

Se m=12 e a=37 il programma stampa?

Se m=11 e a=37 il programma stampa?

Se m=1 e a=18 il programma stampa ?

Se m=15 e a=20 il programma stampa ?

Esercizio

```
b=10; c=15;  
if ((a>b && a>c) || (a<b && a<c))  
    printf("%d",a);  
else  
    printf("%d",b);
```

- Se a=18 il programma stampa
- Se a=12 il programma stampa
- Se a=8 il programma stampa
- Se a=10 il programma stampa

Problema del *dangling else*

```
int main (void)
{
    int x=3,y=4;
    if (x==0)
        if (y<0)
            x=y;
    else
        x=x-y;
}
```

- Quanto vale x?

A quale if è collegato
l'else?

Ogni else è collegato all'if
più vicino
che ne è sprovvisto